

Function Catalog

Generated 2026-07-09

A one-page-per-file index of every function in the codebase: what it is, what it depends on, and what it affects. Not deep documentation — a map.

188

BACKEND FUNCTIONS

92

FRONTEND ENTRIES

59

FILES COVERED

LAYER	FILE	COUNT
1. API layer	main.py	3
1. API layer	server.py	10
1. API layer	auth.py	2
2. Reconciliation engine	orchestrator.py	14
2. Reconciliation engine	forex_api.py	7
2. Reconciliation engine	data_contracts.py	7
3. Agent engine	agent/runner.py	7
3. Agent engine	agent/tools.py	11
3. Agent engine	agent/llm.py	6
3. Agent engine	agent/memory.py	12
3. Agent engine	agent/gate.py	1
3. Agent engine	agent/verifier.py	5
3. Agent engine	agent/anomaly.py	12
3. Agent engine	agent/prompts.py	2
3. Agent engine	agent/__init__.py	0
4. Parsers	statement_parser.py	11
4. Parsers	invoice_parser.py	6
4. Parsers	proof_parser.py	4
4. Parsers	parser_llm.py	3
5. Integrations	myinvois_client.py	9
5. Integrations	accounting_client.py	4
5. Integrations	bank_feed_client.py	12
5. Integrations	bankfeed_state.py	5
6. Seeds & tests	seed_data.py	18
6. Seeds & tests	seed_demo.py	5
6. Seeds & tests	seed_files.py	8
6. Seeds & tests	test_reference_match.py	4
6. Seeds & tests	test_myinvois_map.py	0
6. Seeds & tests	test_accounting_map.py	0
7. Pages	page.tsx	1
7. Pages	layout.tsx	1
7. Pages	login/page.tsx	2
7. Pages	signup/layout.tsx	1
7. Pages	signup/page.tsx	2
7. Pages	uploads/page.tsx	18
7. Pages	audit/page.tsx	5
7. Pages	history/page.tsx	5
7. Pages	settings/page.tsx	6

LAYER	FILE	COUNT
8. Components	components/DashboardPage.tsx	9
8. Components	components/AppShell.tsx	6
8. Components	components/LandingPage.tsx	4
8. Components	components/MeshBackground.tsx	3
9. UI kit	components/ui/ActivityDrawer.tsx	1
9. UI kit	components/ui/Button.tsx	1
9. UI kit	components/ui/Dropzone.tsx	1
9. UI kit	components/ui/EmptyState.tsx	1
9. UI kit	components/ui/Field.tsx	1
9. UI kit	components/ui/PageHeader.tsx	1
9. UI kit	components/ui/Panel.tsx	2
9. UI kit	components/ui/RingProgress.tsx	1
9. UI kit	components/ui/SegmentedControl.tsx	1
9. UI kit	components/ui/Skeleton.tsx	2
9. UI kit	components/ui/StatusPill.tsx	1
9. UI kit	components/ui/Table.tsx	5
9. UI kit	components/ui/Toast.tsx	2
9. UI kit	components/ui/cn.ts	1
9. UI kit	components/ui/index.ts	1
10. lib & contexts	lib/AuthContext.tsx	2
10. lib & contexts	lib/matchActions.ts	2
10. lib & contexts	lib/supabaseClient.ts	1
10. lib & contexts	ThemeContext.tsx	2

Legend

Signature	Reconstructed from source: name, parameters (with defaults/annotations where present), return type. Monospace.
Description	First line of the docstring; if none, a short name-based hint in parentheses — not invented behavior.
Depends on	What this function calls (intra-repo functions, external services/libs) and DB tables/endpoints it reads.
Affects	Who calls this function, and DB tables/endpoints/UI state it writes or drives.
Tags	<code>endpoint</code> = FastAPI route (method + path shown). <code>legacy</code> = superseded by the agent engine but still present. <code>mock-only</code> = the real (non-fixture) code path raises <code>NotImplementedError</code> .

1. API layer

main.py — FastAPI app entrypoint — health, /api/reconcile, /api/job-status; picks agent vs legacy engine.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
async def reconcile(background_tasks: BackgroundTasks, sme_id: str = Depends(get_current_sme_id))	Trigger reconciliation for the authenticated tenant.	agent/runner.run_agent orchestrator.run_reconciliation	—	POST /api/reconcile
async def job_status(sme_id: str = Depends(get_current_sme_id))	Returns the most recent reconciliation job for the authenticated tenant.	reads: reconciliation_job	—	GET /api/job-status
async def health()	(health)	—	—	GET /health

server.py — Upload endpoints (statement/invoice/payment proof) + MyInvois/accounting/bank-feed sync routes.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
async def process_statement(file: UploadFile = File(...), account_id: str None = Form(None), sme_id: str = Depends(get_current_sme_id))	(process statement)	reads: bank_account statement_parser.parse_bank_statement statement_parser.upload_parsed_statement	—	POST /api/upload/statement
async def upload_payment_proof(file: UploadFile = File(...), sme_id: str = Depends(get_current_sme_id))	(upload payment proof)	proof_parser.process_payment_proof	writes: payment_proof writes: storage:proofs	POST /api/upload/payment_proof
async def upload_invoice(file: UploadFile = File(...), sme_id: str = Depends(get_current_sme_id))	(upload invoice)	invoice_parser.process_invoice	writes: invoice writes: storage:invoices	POST /api/upload/invoice
async def myinvois_sync(sme_id: str = Depends(get_current_sme_id))	Pull VALIDATED e-Invoices (both directions) from MyInvois into the invoice table.	reads: myinvois_credential agent/memory.AgentMemory.add myinvois_client.get_document_raw myinvois_client.get_recent_documents myinvois_client.map_document	writes: invoice	POST /api/myinvois/sync
async def accounting_sync(provider: str, sme_id: str = Depends(get_current_sme_id))	Pull sales invoices from an accounting system (provider=autocount sql) into the invoice table.	reads: accounting_credential accounting_client.get_recent_invoices accounting_client.map_invoice	writes: invoice	POST /api/accounting/sync
def _valid_currencies() -> set[str]	Currency codes present in the `currency` table (bank_account/bank_transaction FK to it).	reads: currency	called by server._persist_link_and_accounts called by server.bankfeed_sync	—
def _persist_link_and_accounts(sme_id: str, ident: dict)	Callback-side write (service_role): auto-create a bank_account for the linked Finverse account and store the consent (login_identity token) in bank_feed_link.	server._valid_currencies	writes: bank_account writes: bank_feed_link called by server.bankfeed_callback	—
async def bankfeed_link(sme_id: str = Depends(get_current_sme_id))	Start a Finverse Link session; returns the hosted URL the browser should open.	bank_feed_client.create_link_session bankfeed_state.sign_state	—	POST /api/bankfeed/link

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
async def bankfeed_callback(code: str = '', state: str = '', error: str = '')	PUBLIC — Finverse's browser redirect lands here (no JWT).	bank_feed_client.exchange_ code bankfeed_state.verify_state server._persist_link_and_acc ounts	—	GET /api/bankfeed/callback
async def bankfeed_sync(sme_id: str = Depends(get_current_sme_id))	Pull transactions for every active consent into bank_transaction (existing seam).	reads: bank_feed_link bank_feed_client.get_transa ctions server._valid_currencies statement_parser.upload_pa rsed_statement	—	POST /api/bankfeed/sync

auth.py — JWT auth dependency — resolves sme_id from the Supabase session for every protected route.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _bearer_token(authorization: str None) -> str	(bearer token)	—	called by auth.get_current_sme_id	—
async def get_current_sme_id(authorization: str None = Header(default=None)) -> str	FastAPI dependency: validate the Bearer JWT and resolve the caller's sme_id.	reads: sme auth._bearer_token	—	—

2. Reconciliation engine

orchestrator.py – real / shared — Core recon primitives (fetch/write/log) shared by both engines, plus the legacy linear pipeline.

(real/shared functions)

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def _log(db: Client, job_id: str, event_type: str, message: str, metadata: dict = None)</code>	(log)	—	writes: reconciliation_log called by agent/runner._fail_or_fallback called by orchestrator._complete_job called by orchestrator._reference_pre_match called by orchestrator._start_job called by orchestrator._write_match called by orchestrator.run_reconciliation	—
<code>def _start_job(db: Client, sme_id: str) -> str</code>	(start job)	orchestrator_log	writes: reconciliation_job called by agent/runner.run_agent called by orchestrator.run_reconciliation	—
<code>def _fetch_invoices(db: Client, sme_id: str) -> list[InvoiceForMatching]</code>	(fetch invoices)	reads: invoice	called by agent/runner._finalise called by agent/runner.run_agent called by agent/tools.list_invoices called by orchestrator._reference_pre_match called by orchestrator.run_reconciliation	—
<code>def _fetch_transactions(db: Client, sme_id: str, days: int = LOOKBACK_DAYS) -> list[TransactionForMatching]</code>	(fetch transactions)	reads: bank_transaction	called by agent/runner.run_agent called by agent/tools.find_candidates called by agent/tools.list_transactions called by orchestrator._reference_pre_match called by orchestrator.run_reconciliation	—
<code>def _fetch_proofs(db: Client, sme_id: str) -> list[ProofForMatching]</code>	(fetch proofs)	reads: payment_proof	called by agent/tools.list_proofs called by orchestrator._reference_pre_match called by orchestrator.run_reconciliation	—
<code>def _norm_ref(s: str None) -> str None</code>	Normalise a reference for exact comparison: drop spaces/dashes, uppercase.	—	called by orchestrator._reference_pairs called by test_reference_match.main	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _reference_pairs(invoices: list[InvoiceForMatching], transactions: list[TransactionForMatching], proofs: list[ProofForMatching]) -> list[tuple[InvoiceForMatching, TransactionForMatching, ProofForMatching None]]	Pure: find (invoice, transaction, proof?) triples that share an exact DuitNow/FPX reference.	agent/memory.AgentMemory.add orchestrator._norm_ref	called by orchestrator._reference_pre_match called by test_reference_match.main	—
def _reference_prematch(db: Client, job_id: str, sme_id: str) -> tuple[int, set[str]]	Commit exact-reference matches through the same gate + writer the LLM path uses, BEFORE any model runs.	agent/gate.decide agent/memory.AgentMemory.add forex_api.get_rate orchestrator._fetch_invoices orchestrator._fetch_proofs orchestrator._fetch_transactions orchestrator._log orchestrator._reference_pairs orchestrator._write_match	called by agent/runner.run_agent called by orchestrator.run_reconciliation	—
def _write_match(db: Client, job_id: str, proposal: MorpheusMatchProposal, invoice: InvoiceForMatching, transaction: TransactionForMatching, rate_id: str, rate: float, force_status: MatchStatus = None) -> ReconciliationMatchInsert	(write match)	data_contracts.ReconciliationMatchInsert.build orchestrator._log	writes: bank_transaction writes: invoice writes: reconciliation_match called by agent/tools.propose_match called by orchestrator._reference_pre_match called by orchestrator.run_reconciliation	—
def _complete_job(db: Client, job_id: str, matched: int, unmatched: int)	(complete job)	orchestrator._log	writes: reconciliation_job called by agent/runner._finalise called by orchestrator.run_reconciliation	—

orchestrator.py – legacy (linear pipeline) — Superseded by the agent engine (agent/runner.py) but kept for USE_AGENT=false fallback.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _build_morpheus_prompt(job_input: OrchestratorJobInput, rates: dict) -> str	(build morpheus prompt)	—	called by orchestrator.run_reconciliation	legacy
def _call_morpheus(prompt: str) -> list[MorpheusMatchProposal]	(call morpheus)	httpx (HTTP client)	called by orchestrator.run_reconciliation	legacy
def _write_recommendation(db: Client, sme_id: str, job_id: str, unmatched_invoices: list[InvoiceForMatching])	(write recommendation)	—	writes: recommendation called by orchestrator.run_reconciliation	legacy

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def run_reconciliation(sme_id: str) -> dict	(run reconciliation)	agent/memory.AgentMemory.add forex_api.get_rates_batch orchestrator._build_morphheus_prompt orchestrator._call_morpheus orchestrator._complete_job orchestrator._fetch_invoices orchestrator._fetch_proofs orchestrator._fetch_transactions orchestrator._log orchestrator._reference_pre_match orchestrator._start_job orchestrator._write_match orchestrator._write_recommendation	writes: invoice writes: reconciliation_job called by agent/runner._fail_or_fallback called by main.reconcile	legacy

forex_api.py — FX rate resolution — cache-first against `exchange\_rate`, Frankfurter fallback.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _to_utc_iso(d: date) -> str	Convert a date to midnight UTC ISO string for TIMESTAMPTZ comparison.	—	called by forex_api.check_cache called by forex_api.insert_rate	—
def _log(supabase: Client, job_id: str, event_type: str, message: str, metadata: dict = None)	Write one row to reconciliation_log.	—	writes: reconciliation_log called by forex_api.get_rate called by forex_api.get_rates_batch	—
def _check_cache(supabase: Client, query: ForexCacheQuery) -> tuple[str, float] None	Return (rate_id, rate) if row exists in exchange_rate, else None.	reads: exchange_rate forex_api._to_utc_iso	called by forex_api.get_rate called by forex_api.get_rates_batch	—
def _call_frankfurter(req: FrankfurterRequest) -> list[FrankfurterResponse]	(call frankfurter)	httpx (HTTP client) data_contracts.FrankfurterRequest.to_url	called by forex_api.get_rate called by forex_api.get_rates_batch	—
def _insert_rate(supabase: Client, api_response: FrankfurterResponse, txn_date: date) -> tuple[str, float]	Insert one exchange_rate row.	reads: exchange_rate data_contracts.ExchangeRateInsert.from_api_response forex_api._to_utc_iso	writes: exchange_rate called by forex_api.get_rate called by forex_api.get_rates_batch	—
def get_rate(supabase: Client, from_currency: str, to_currency: str, txn_date: date, job_id: str) -> tuple[str, float]	Return (rate_id, rate) for one currency pair on txn_date.	forex_api._call_frankfurter forex_api._check_cache forex_api._insert_rate forex_api._log	called by agent/tools.find_candidates called by agent/tools.get_fx_rate called by agent/tools.propose_match called by orchestrator._reference_pre_match	—
def get_rates_batch(supabase: Client, queries: list[ForexCacheQuery], job_id: str) -> dict[tuple[str, str, str], tuple[str, float]]	Return a dict of (from_currency, to_currency, date_str) → (rate_id, rate) for a list of ForexCacheQuery objects.	forex_api._call_frankfurter forex_api._check_cache forex_api._insert_rate forex_api._log	called by orchestrator.run_reconciliation	legacy

data_contracts.py — Pydantic v2 schemas validating parsed invoice/statement/proof data before it's trusted.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
TransactionSchema.currency_must_be_iso(cls, v: str) -> str	(currency must be iso)	—	—	—
TransactionSchema.date_must_be_iso(cls, v: str) -> str	(date must be iso)	—	—	—
ChutesParserOutput.currency_uppercase(cls, v: str None) -> str None	(currency uppercase)	—	—	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
FrankfurterRequest.to_url(self) -> str	(to url)	—	called by forex_api_call_frankfurter	—
ExchangeRateInsert.model_post_init(self, __context: Any) -> None	(model post init)	—	—	—
ExchangeRateInsert.from_api_response(cls, response: FrankfurterResponse, transaction_date: date) -> 'ExchangeRateInsert'	(from api response)	—	called by forex_api_insert_rate	—
ReconciliationMatchInsert.build(cls, job_id: str, proposal: MorpheusMatchProposal, invoice: InvoiceForMatching, transaction: TransactionForMatching, rate: float, rate_id: Optional[str] = None, threshold: float = 0.75, force_status: Optional['MatchStatus'] = None) -> 'ReconciliationMatchInsert'	Build the row written to reconciliation_match.	—	called by orchestrator._write_match called by seed_demo.main called by seed_files.main	—

3. Agent engine

agent/runner.py — Agentic reconciliation loop — the default engine (USE_AGENT=true): tool calls, verify, finalize.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def _short_args(args: dict) -> str</code>	(short args)	—	called by agent/runner._run_batch	—
<code>def _job_cancelled(db, job_id: str) -> bool</code>	(job cancelled)	reads: reconciliation_job	called by agent/runner._run_batch	—
<code>def run_agent(sme_id: str, mode: str = 'auto') -> dict</code>	(run agent)	agent/llm.get_mode agent/memory.AgentMemory.persist agent/memory.load_learned agent/runner._fail_or_fallback agent/runner._finalise agent/runner._run_batch agent/runner._worker agent/tools.scan_anomalies agent/verifier.run_verification orchestrator._fetch_invoices orchestrator._fetch_transactions orchestrator._reference_pre_match orchestrator._start_job	writes: reconciliation_job called by main.reconcile	—
<code>def _worker(item)</code>	(worker)	agent/runner._run_batch	called by agent/runner.run_agent	—
<code>def _run_batch(db, job_id, sme_id, ctx, batch, batch_no, total, llm_mode, learned, mode, overall_start) -> str</code>	One focused sub-loop over a small batch of invoices.	agent/llm.step agent/llm.tool_result_message agent/memory.AgentMemory.add agent/memory.AgentMemory.add_system agent/memory.AgentMemory.add_user agent/memory.AgentMemory.persist agent/prompts.build_batch_goal agent/prompts.build_system_prompt agent/runner._job_cancelled agent/runner._short_args agent/tools.openai_tools_params agent/tools.react_tool_descriptions	called by agent/runner._worker called by agent/runner.run_agent	—
<code>def _finalise(db, job_id: str, sme_id: str, ctx: dict, status: str = 'completed') -> dict</code>	Sweep still-unmatched invoices and write the job to a terminal state.	orchestrator._complete_job orchestrator._fetch_invoices	writes: invoice writes: reconciliation_job called by agent/runner.run_agent	—
<code>def _fail_or_fallback(db, job_id: str, sme_id: str, exc: Exception, did_progress: bool) -> dict</code>	Model unreachable: optionally run the legacy pipeline; else mark failed.	orchestrator._log orchestrator.run_reconciliation	writes: reconciliation_job called by agent/runner.run_agent	—

agent/tools.py — Tool surface exposed to the LLM (list_invoices/list_transactions/propose_match/etc.) + tool-schema builders.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def list_invoices(db, job_id, sme_id, ctx, mem, mode, status_filter = None, **_)</code>	(list invoices)	agent/memory.AgentMemory.persist orchestrator._fetch_invoices	—	—
<code>def list_transactions(db, job_id, sme_id, ctx, mem, mode, lookback_days = None, **_)</code>	(list transactions)	agent/memory.AgentMemory.persist orchestrator._fetch_transactions	—	—
<code>def list_proofs(db, job_id, sme_id, ctx, mem, mode, **_)</code>	(list proofs)	agent/memory.AgentMemory.persist orchestrator._fetch_proofs	—	—
<code>def find_candidates(db, job_id, sme_id, ctx, mem, mode, invoice_id = None, limit = 8, **_)</code>	Return the most likely UNMATCHED transactions for one invoice — a deterministic shortlist scored by amount (post-FX) + date proximity + counterparty/descript...	agent/memory.AgentMemory.persist agent/verifier.tokens forex_api.get_rate orchestrator._fetch_transactions	—	—
<code>def get_fx_rate(db, job_id, sme_id, ctx, mem, mode, from_currency = None, to_currency = None, on_date = None, **_)</code>	(get fx rate)	forex_api.get_rate	—	—
<code>def verify_matches(db, job_id, sme_id, ctx, mem, mode, **_)</code>	Re-examine this run's matches without mutating anything.	agent/verifier.run_verification	—	—
<code>def scan_anomalies(db, job_id, sme_id, ctx, mem, mode, **_)</code>	Scan the workspace for fraud/anomaly signals (duplicate billing, payments from the wrong party, changed bank details, amount outliers) without mutating anything...	—	called by agent/runner.run_agent	—
<code>def propose_match(db, job_id, sme_id, ctx, mem, mode, invoice_id = None, transaction_id = None, confidence = None, rationale = None, proof_id = None, **_)</code>	(propose match)	agent/gate.decide agent/memory.AgentMemory.add agent/memory.AgentMemory.persist forex_api.get_rate orchestrator.write_match	—	—
<code>def _finish_placeholder(*_args, **_kwargs)</code>	(finish placeholder)	—	—	—
<code>def openai_tools_param(names: list[str] None = None) -> list[dict]</code>	(openai tools param)	—	called by agent/runner.run_batch	—
<code>def react_tool_descriptions(names: list[str] None = None) -> str</code>	(react tool descriptions)	—	called by agent/runner.run_batch	—

agent/llm.py — Model-agnostic chat step wrapper around Morpheus (OpenAI-compatible) for the agent loop.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def get_mode() -> str</code>	(get mode)	agent/llm._probe_native	called by agent/llm.step called by agent/llm.tool_result_message called by agent/runner.run_agent	—
<code>def _probe_native() -> str</code>	Force a trivial tool call; if the gateway returns one, native works.	—	called by agent/llm.get_mode	—
<code>def step(messages: list[dict], tools_param: list[dict] None) -> StepResult</code>	(step)	agent/llm._step_native agent/llm._step_react agent/llm.get_mode	called by agent/runner.run_batch	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _step_native(messages: list[dict], tools_param: list[dict] None) -> StepResult	(step native)	—	called by agent/llm.step	—
def _step_react(messages: list[dict]) -> StepResult	(step react)	parser_llm._strip_fences	called by agent/llm.step	—
def tool_result_message(tool_call: ToolCall, result: dict) -> dict	Format a tool's return value as the next message, per mode.	agent/llm.get_mode	called by agent/runner._run_batch	—

agent/memory.py — Per-job conversation memory + cross-job learned memory (accepted/rejected patterns).

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
AgentMemory.__init__(self, db: Client, job_id: str)	(init)	—	—	—
AgentMemory.add_system(self, content: str) -> None	(add system)	—	called by agent/runner._run_batch	—
AgentMemory.add_user(self, content: str) -> None	(add user)	—	called by agent/runner._run_batch	—
AgentMemory.add(self, message: dict) -> None	Append a raw message dict (assistant turn or tool/observation result).	—	called by agent/anomaly._check_ban k_detail_change called by agent/memory._load_blocklist called by agent/runner._run_batch called by agent/tools.propose_match called by orchestrator._reference_pairs called by orchestrator._reference_pre_match called by orchestrator.run_reconciliation called by server.myinvois_sync	—
AgentMemory.messages(self) -> list[dict]	The thread sent to the model, trimmed to a character budget.	—	—	—
AgentMemory.persist(self, event_type: str, message: str, metadata: dict = None, phase: str = None) -> None	(persist)	—	called by agent/anomaly.scan_anomalies called by agent/runner._run_batch called by agent/runner.run_agent called by agent/tools.find_candidates called by agent/tools.list_invoices called by agent/tools.list_proofs called by agent/tools.list_transactions called by agent/tools.propose_match called by agent/verifier.run_verification	—
def _fmt_variance(v) -> str	(fmt variance)	—	called by agent/memory._format_summary	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _trim(text, width: int = 48) -> str	(trim)	—	called by agent/memory_load_positives	—
def load_learned(db: Client, sme_id: str) -> dict	Long-term learned memory: distil priors from this workspace's history.	reads: reconciliation_job agent/memory_format_summary agent/memory_load_blocklist agent/memory_load_positives	called by agent/runner.run_agent	—
def _load_positives(db: Client, job_ids: list[str]) -> list[dict]	Human-confirmed matches, aggregated into one exemplar per counterparty.	reads: bank_transaction reads: reconciliation_match agent/memory_trim	called by agent/memory.load_learned	—
def _load_blocklist(db: Client, job_ids: list[str]) -> list[dict]	Recent rejected pairings, deduped, as a small avoid-list.	reads: reconciliation_log reads: reconciliation_match agent/memory.AgentMemory.add	called by agent/memory.load_learned	—
def _format_summary(exemplars: list[dict], blocklist: list[dict]) -> str	Deterministic prose block.	agent/memory_fmt_variance	called by agent/memory.load_learned	—

agent/gate.py — Confidence-gate decision: auto-accept vs pending_review threshold.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def decide(confidence: float, variance_pct: float, converted_amount: float, auto_count: int) -> tuple[Verdict, list[str]]	Return (verdict, reasons).	—	called by agent/tools.propose_match called by orchestrator._reference_pre_match called by test_reference_match.main	—

agent/verifier.py — Post-hoc pass catching invoices the agent missed / verifying proposed matches.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def tokens(text: str) -> set[str]	Lowercase alphanumeric tokens of length >=3, minus generic company words.	—	called by agent/anomaly_check_beneficiary_mismatch called by agent/tools.find_candidates called by agent/verifier.run_verification	—
def run_verification(db, job_id, sme_id, ctx, mem, *, apply: bool) -> dict	Inspect every match this job wrote and (when apply=True) downgrade the risky ones.	reads: reconciliation_match agent/memory.AgentMemory.persist agent/verifier_downgrade agent/verifier_find_missed_txn agent/verifier_tokens agent/verifier.finding	called by agent/runner.run_agent called by agent/tools.verify_matches	—
def finding(code, severity, detail, **extra)	(finding)	—	called by agent/verifier.run_verification	—
def _find_missed_txn(inv, txn_map, used_txn_ids, ctx) -> str None	Return a candidate transaction_id whose converted amount lands in the auto band, or None.	—	called by agent/verifier.run_verification	—
def _downgrade(db, f: dict) -> bool	Flip a still-auto match to pending_review (+ invoice to partial).	—	writes: invoice writes: reconciliation_match called by agent/anomaly.scan_anomalies called by agent/verifier.run_verification	—

agent/anomaly.py — Heuristic anomaly scans: duplicate invoices, beneficiary mismatch, bank-detail change, amount outliers.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _parse_date(v)	(parse date)	—	called by agent/anomaly._check_duplicate_invoices	—
def scan_anomalies(db, job_id, sme_id, ctx, mem, *, apply: bool) -> dict	Run the four deterministic checks, persist findings, and (when apply=True) escalate high-severity fraud signals on this run's auto-commits.	agent/anomaly._counterparty_history agent/anomaly._fetch_invoices agent/anomaly._fetch_proofs agent/anomaly._job_auto_matches agent/memory.AgentMemory.persist agent/verifier._downgrade	—	—
def _fetch_invoices(db, sme_id) -> list[dict]	(fetch invoices)	reads: invoice	called by agent/anomaly.scan_anomalies	—
def _fetch_proofs(db, sme_id) -> list[dict]	(fetch proofs)	reads: payment_proof	called by agent/anomaly.scan_anomalies	—
def _counterparty_history(db, sme_id) -> dict[str, list[float]]	Map counterparty_name -> list of past matched converted amounts, for outlier baselines.	reads: reconciliation_job reads: reconciliation_match	called by agent/anomaly.scan_anomalies	—
def _job_auto_matches(db, job_id) -> dict[str, str]	invoice_id -> match_id for matches THIS job auto-committed (escalation targets).	reads: reconciliation_match	called by agent/anomaly.scan_anomalies	—
def _check_duplicate_invoices(invoices, proofs, history) -> list[dict]	(check duplicate invoices)	agent/anomaly._emit_dup agent/anomaly._parse_date	—	—
def _emit_dup(out: list[dict], cluster: list[dict], amount: float) -> None	(emit dup)	—	called by agent/anomaly._check_duplicate_invoices	—
def _check_beneficiary_mismatch(invoices, proofs, history) -> list[dict]	(check beneficiary mismatch)	agent/verifier.tokens	—	—
def _check_bank_detail_change(invoices, proofs, history) -> list[dict]	(check bank detail change)	agent/anomaly.identity agent/memory.AgentMemory.add	—	—
def identity(pdata: dict) -> str None	(identity)	—	called by agent/anomaly._check_bank_detail_change	—
def _check_amount_outlier(invoices, proofs, history) -> list[dict]	(check amount outlier)	—	—	—

agent/prompts.py — System prompt builder for the agent loop.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def build_batch_goal(invoices: list, batch_no: int, total_batches: int) -> str	(build batch goal)	—	called by agent/runner._run_batch	—
def build_system_prompt(mode: str, learned_summary: str, tool_descriptions: str, batched: bool = False) -> str	(build system prompt)	—	called by agent/runner._run_batch	—

agent/__init__.py — Package marker — no functions.

No top-level functions extracted (file has no module-level def).

4. Parsers

statement_parser.py — CSV/XLSX bank statement parsing into `bank\_transaction` rows (account-scoped dedup).

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _detect_encoding(file_path: str) -> str	(detect encoding)	—	called by statement_parser.parse_bank_statement	—
def _resolve_column(columns: list[str], aliases: set[str]) -> str None	(resolve column)	—	called by statement_parser.merge_debit_credit called by statement_parser.parse_bank_statement	—
def _clean_amount(raw: Any) -> float	(clean amount)	—	called by statement_parser.cell called by statement_parser.merge_debit_credit called by statement_parser.parse_bank_statement	—
def _parse_date(raw: Any) -> str	(parse date)	—	called by statement_parser.parse_bank_statement	—
def _normalise_description(raw: str) -> str	(normalise description)	—	called by bank_feed_client.map_transaction called by statement_parser.parse_bank_statement	—
def _extract_reference(raw: str) -> str None	Pull a DuitNow/FPX reference token out of a bank narrative (e.g.	—	called by bank_feed_client.map_transaction called by statement_parser.parse_bank_statement called by test_reference_match.main	—
def _fingerprint(date_iso: str, description: str, amount: float, occurrence: int) -> str	(fingerprint)	—	called by statement_parser.parse_bank_statement	—
def _merge_debit_credit(df: pd.DataFrame) -> pd.DataFrame	(merge debit credit)	pandas statement_parser.cell statement_parser.clean_amount statement_parser.resolve_column	called by statement_parser.parse_bank_statement	—
def _cell(x: Any) -> float	(cell)	pandas statement_parser.clean_amount	called by statement_parser.merge_debit_credit	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def parse_bank_statement(file_path: str, local_currency: str, date_from: date None = None, date_to: date None = None) -> dict[str, Any]	(parse bank statement)	pandas statement_parser_clean_amount statement_parser_detect_encoding statement_parser_extract_reference statement_parser_fingerprint statement_parser_merge_debit_credit statement_parser_normalise_description statement_parser_parse_date statement_parser_resolve_column	called by server.process_statement	—
def upload_parsed_statement(parsed_ result: dict[str, Any], statement_id: str, supabase: Any, account_id: str None = None, sme_id: str None = None, file_type: str = 'csv')	(upload parsed statement)	reads: bank_account	writes: bank_statement writes: bank_transaction called by server.bankfeed_sync called by server.process_statement	—

invoice_parser.py — PDF/image invoice parsing via PyMuPDF/Tesseract + Morpheus structuring into `invoice`.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
ParsedInvoiceData.currency_uppercase(cls, v: str None) -> str None	(currency uppercase)	—	—	—
def extract_invoice_image(input_data: InvoiceParserInput) -> InvoiceParserOutput	(extract invoice image)	reads: storage:invoices parser_llm.ocr_image parser_llm.structure	called by invoice_parser.process_invoice	—
def extract_invoice_pdf(input_data: InvoiceParserInput) -> InvoiceParserOutput	(extract invoice pdf)	PyMuPDF (fitz) reads: storage:invoices parser_llm.structure	called by invoice_parser.process_invoice	—
def _validate_parsed_invoice(data: ParsedInvoiceData) -> str None	Return an error message if the parsed data would violate a DB constraint, else None.	—	called by invoice_parser.process_invoice	—
def process_invoice(invoice_id: str, file_path: str, file_type: str)	Handle the lifecycle of an uploaded invoice.	invoice_parser.do_update invoice_parser.validate_parsed_invoice invoice_parser.extract_invoice_image invoice_parser.extract_invoice_pdf	writes: invoice called by server.upload_invoice	—
def _do_update(payload)	(do update)	—	writes: invoice called by invoice_parser.process_invoice	—

proof_parser.py — PDF/image payment-proof parsing into `payment\_proof`.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def extract_proof_image(input_data: ChutesParserInput) -> ChutesParserOutput	(extract proof image)	reads: storage:proofs parser_llm.ocr_image parser_llm.structure	called by proof_parser.process_payment_proof	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def extract_proof_pdf(input_data: ChutesParserInput) -> ChutesParserOutput	(extract proof pdf)	PyMuPDF (fitz) reads: storage:proofs parser_llm.structure	called by proof_parser.process_paym ent_proof	—
def _validate_parsed_proof(output: ChutesParserOutput) -> str None	Return an error message if the parsed data would violate a DB constraint, else None.	—	called by proof_parser.process_paym ent_proof	—
def process_payment_proof(proof_id: str, file_path: str, file_type: str)	Handle the lifecycle of an uploaded payment proof.	proof_parser.validate_parse d_proof proof_parser.extract_proof_i mage proof_parser.extract_proof_ pdf	writes: payment_proof called by server.upload_payment_pro of	—

parser_llm.py — Shared Morpheus text->JSON structuring call used by invoice/proof parsers.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def ocr_image(file_bytes: bytes) -> str	Extract text from an image via Tesseract.	pytesseract (Tesseract OCR)	called by invoice_parser.extract_invoic e_image called by proof_parser.extract_proof_i mage	—
def _strip_fences(raw: str) -> str	Remove ```json ...	—	called by agent/llm._step_react called by parser_llm.structure	—
def structure(document_text: str, instruction: str) -> dict	Send extracted text to Morpheus and return the parsed JSON object.	parser_llm._strip_fences	called by invoice_parser.extract_invoic e_image called by invoice_parser.extract_invoic e_pdf called by proof_parser.extract_proof_i mage called by proof_parser.extract_proof_ pdf	—

5. Integrations

myinvois_client.py — LHDN MyInvois OAuth2 client + UBL document mapping into `invoice` rows.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def get_token(creds: dict) -> str</code>	(get token)	htpx (HTTP client)	called by myinvois_client_auth_headers	—
<code>def _auth_headers(creds: dict) -> dict</code>	(auth headers)	myinvois_client.get_token	called by myinvois_client.get_document_raw called by myinvois_client.get_recent_documents	—
<code>def get_recent_documents(creds: dict, direction: str) -> list[dict]</code>	List Valid documents (last 31 days) for a direction ('Sent' 'Received').	htpx (HTTP client) myinvois_client_auth_headers	called by server.myinvois_sync	—
<code>def get_document_raw(creds: dict, uuid: str) -> dict</code>	Fetch the raw UBL 2.1 JSON for a document (carries DocumentCurrencyCode + totals, which the recent/details endpoints omit — MYR only).	htpx (HTTP client) myinvois_client_auth_headers	called by server.myinvois_sync	—
<code>def _ubl(node, *path)</code>	Walk MyInvois UBL JSON, which nests every field as [{"_": value, ...}].	—	called by myinvois_client_party_name called by myinvois_client.map_document	—
<code>def _party_name(inv, party_key: str)</code>	(party name)	myinvois_client_ubl myinvois_client_ubl_node	called by myinvois_client.map_document	—
<code>def _ubl_node(node, *path)</code>	Like _ubl but returns the sub-node (dict/list), not the leaf scalar.	—	called by myinvois_client_party_name	—
<code>def map_document(ubl: dict, direction: str, sme_id: str, uuid: str) -> dict</code>	UBL 2.1 JSON -> an `invoice` row.	myinvois_client_party_name myinvois_client_ubl	called by server.myinvois_sync	—
<code>def _mk_ubl(inv_id, currency, amount, issue, due, supplier, customer)</code>	(mk ubl)	—	—	—

accounting_client.py — Mock AutoCount/SQL Account connector — fixture invoices mapped to `invoice` rows.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
<code>def get_recent_invoices(creds: dict) -> list[dict]</code>	List sales invoices for the tenant's configured provider.	—	called by server.accounting_sync	mock-only
<code>def _pick(doc: dict, *keys)</code>	First present, non-empty value among candidate keys.	—	called by accounting_client.map_invoice	—
<code>def map_invoice(doc: dict, sme_id: str, provider: str) -> dict</code>	Mock fixture dict -> an `invoice` row.	accounting_client_pick	called by server.accounting_sync	—
<code>def _inv(doc_no, debtor, currency, total, doc_date, due)</code>	(inv)	—	—	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _env() -> str	(env)	—	called by bank_feed_client._customer_token called by bank_feed_client._host called by bank_feed_client.create_link_session called by bank_feed_client.exchange_code called by bank_feed_client.get_transactions	—
def _host() -> str	(host)	bank_feed_client._env	called by bank_feed_client._customer_token called by bank_feed_client.create_link_session called by bank_feed_client.exchange_code called by bank_feed_client.get_transactions	—
def _redirect_uri() -> str	(redirect uri)	—	called by bank_feed_client._demo called by bank_feed_client.create_link_session called by bank_feed_client.exchange_code	—
def _frontend_url() -> str	(frontend url)	—	—	—
def _customer_token() -> str	(customer token)	httpx (HTTP client) bank_feed_client._env bank_feed_client._host	called by bank_feed_client.create_link_session called by bank_feed_client.exchange_code	—
def create_link_session(state: str) -> str	Return the URL the user's browser should visit to authorize their bank.	httpx (HTTP client) bank_feed_client._customer_token bank_feed_client._env bank_feed_client._host bank_feed_client._redirect_uri	called by bank_feed_client._demo called by server.bankfeed_link	—
def exchange_code(code: str) -> dict	code -> {finverse_login_id, access_token, refresh_token, token_expires_at, accounts:[...]}	httpx (HTTP client) bank_feed_client._customer_token bank_feed_client._env bank_feed_client._host bank_feed_client._redirect_uri	called by server.bankfeed_callback	—
def map_transaction(doc: dict) -> dict None	Finverse transaction -> a TransactionSchema-shaped row, or None to drop it.	statement_parser.extract_reference statement_parser.normalise_description	called by bank_feed_client.get_transactions	—
def _to_parsed_result(rows: list[dict]) -> dict	(to parsed result)	—	called by bank_feed_client.get_transactions	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def get_transactions(link: dict) -> dict	Pull all transactions for a stored consent -> parser blob for upload_parsed_statement.	httpx (HTTP client) bank_feed_client.env bank_feed_client.host bank_feed_client.to_parsed_result bank_feed_client.map_transactions	called by bank_feed_client_demo called by server.bankfeed_sync	—
def _mt(txn_id, day, value, desc, currency = 'MYR', pending = False)	(mt)	—	—	—
def _demo()	Self-check: mock get_transactions yields exactly the keys the ingestion seam consumes, the pending row is dropped, and signs map correctly.	bank_feed_client.redirect_uri bank_feed_client.create_link_session bank_feed_client.get_transactions	—	—

bankfeed_state.py — HMAC-signed `state` token carrying sme_id across the JWT-less OAuth redirect.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _secret() -> bytes	(secret)	—	called by bankfeed_state_sign	—
def _sign(payload: str) -> str	(sign)	bankfeed_state_secret	called by bankfeed_state_demo called by bankfeed_state.sign_state called by bankfeed_state.verify_state	—
def sign_state(sme_id: str) -> str	Return a compact, tamper-evident token carrying sme_id + issue time.	bankfeed_state_sign	called by bankfeed_state_demo called by server.bankfeed_link	—
def verify_state(state: str) -> str	Return the sme_id from a valid, unexpired state, else raise ValueError.	bankfeed_state_sign	called by bankfeed_state_demo called by server.bankfeed_callback	—
def _demo()	Self-check for the security path: roundtrip, tamper reject, expiry reject.	bankfeed_state_sign bankfeed_state.sign_state bankfeed_state.verify_state	—	—

6. Seeds & tests

seed_data.py — Deterministic (uuid5) demo-data builder — 4 tenants, accounts, history, current docs.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def uid(*parts) -> str	(uid)	—	called by seed_data.accounts called by seed_data.build_history called by seed_data.inv called by seed_data.proof called by seed_data.build	—
def _utc_iso(d: date) -> str	(utc iso)	—	called by seed_data.build_history called by seed_data.proof called by seed_data.add_fx called by seed_data.build	—
def txid(account_id: str, date_iso: str, description: str, signed_amount: float, occurrence: int = 0) -> str	Replicate statement_parser's account-scoped transaction id so a seeded transaction is identical to one produced by uploading the same statement.	—	called by seed_data.match_txn called by seed_data.noise_txn	—
def rate_for(frm: str, to: str) -> float	(rate for)	—	called by seed_data.add_fx called by seed_data.build called by seed_data.conv	—
def _accounts(sme)	(accounts)	seed_data.uid	called by seed_data.build	—
def _noise_txn(account_id, ccy, rng, d)	(noise txn)	seed_data.txid	called by seed_data.build	—
def _match_txn(account_id, ccy, d, desc, credit_amt, reference = None)	(match txn)	seed_data.txid	called by seed_data.build_history called by seed_data.build called by seed_data.emit	—
def _inv(sme_id, num, cp, ccy, amt, idate, due, *, status = 'pending', error = None, tag = None, source = 'upload', source_ref = None, myinvois_uuid = None, direction = None)	(inv)	seed_data.uid	called by seed_data.build_history called by seed_data.build called by seed_data.emit	—
def _proof(sme_id, invoice_id, amt, ccy, pdate, ref, *, sender = None, bank = None, account_number = None, swift = None, status = 'completed', error = None, tag = None, uploaded = None, rail = None, recipient_reference = None)	(proof)	seed_data._utc_iso seed_data.uid	called by seed_data.build	—
def _build_history(sme, primary, rng)	A completed reconciliation job: matched invoices/transactions + match rows.	seed_data.inv seed_data.match_txn seed_data._utc_iso seed_data.uid	called by seed_data.build	—

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def build()	Return the resolved tenant dataset + global FX + the ground-truth expectations.	seed_data.accounts seed_data.build_history seed_data.due seed_data.inv seed_data.match_txn seed_data.noise_txn seed_data.proof seed_data.utc_iso seed_data.add_fx seed_data.assign_source seed_data.conv seed_data.credit seed_data.emit seed_data.nextcp seed_data.rate_for seed_data.uid	—	—
def add_fx(frm, to, d)	(add fx)	seed_data.utc_iso seed_data.rate_for	called by seed_data.build called by seed_data.emit	—
def _due(idate)	(due)	—	called by seed_data.build called by seed_data.emit	—
def emit(num, cp, ccy, amt, idate, *, tier, expected_status, should_match, source = 'upload', source_ref = None, myinvois_uuid = None, direction = None, credits = None, proof = None, tag = None, note = '', scored = True)	Materialise one invoice (+ optional matching credit(s) + proof) and record its ground-truth expectation.	seed_data.due seed_data.inv seed_data.match_txn seed_data.add_fx	called by seed_data.build	—
def credit(account, idate, desc, amount, offset = 2, ref = None, day = None)	(credit)	—	called by seed_data.build	—
def conv(ccy, amt, variance_pct = 0.0)	(conv)	seed_data.rate_for	called by seed_data.build	—
def assign_source()	(assign source)	—	called by seed_data.build	—
def nextcp()	(nextcp)	—	called by seed_data.build	—

seed_demo.py — Destructive+idempotent reseed: wipes tenant data, provisions the 4 demo Supabase Auth logins.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def wipe_all() -> None	(wipe all)	—	called by seed_demo.main	—
def _list_users() -> list	(list users)	—	called by seed_demo.prune_and_provision_users	—
def prune_and_provision_users(target_emails: set[str]) -> dict[str, str]	Delete auth users not in the demo set; ensure each demo email exists with the documented password and a confirmed email.	seed_demo._list_users	called by seed_demo.main	—
def _strip(row: dict) -> dict	Drop internal helper keys (anything starting with '_') and nested children.	—	called by seed_demo.main	—
def main() -> None	(main)	reads: sme data_contracts.ReconciliationMatchInsert.build seed_demo._strip seed_demo.prune_and_provision_users seed_demo.wipe_all	writes: accounting_credential writes: bank_account writes: bank_feed_link writes: bank_statement writes: bank_transaction writes: exchange_rate writes: invoice writes: myinvois_credential writes: payment_proof writes: reconciliation_job writes: reconciliation_match writes: sme	—

seed_files.py — Regenerates the `test\_files/` upload fixtures + `sme\_infos` manifest.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _render_lines(lines: list[str], path: str, as_png: bool) -> None	(render lines)	PyMuPDF (fitz)	called by seed_files.main	—
def _invoice_lines(inv: dict, sme: dict) -> list[str]	(invoice lines)	—	called by seed_files.main	—
def _proof_lines(p: dict) -> list[str]	(proof lines)	—	called by seed_files.main	—
def _write_statement(stmt: dict, path: str) -> None	(write statement)	pandas	called by seed_files.main	—
def _write_invalid_files(root: str) -> list[dict]	Deliberately broken uploads so the demo can show honest parse-failure states.	—	called by seed_files.main	—
def _acct_label(accounts: list[dict], account_id: str) -> dict	(acct label)	—	called by seed_files.main	—
def main() -> None	(main)	data_contracts.ReconciliationMatchInsert.build seed_files._acct_label seed_files._invoice_lines seed_files._proof_lines seed_files._readme seed_files._render_lines seed_files._write_invalid_files seed_files._write_statement	—	—
def _readme(data, counts, invalid_entries) -> str	(readme)	—	called by seed_files.main	—

test_reference_match.py — Self-check for the DuitNow/FPX exact-reference recon pre-pass.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	TAGS
def _inv(num, amount, iid = None, currency = 'MYR')	(inv)	—	called by test_reference_match.main	—
def _txn(ref, credit, tid = None, currency = 'MYR')	(txn)	—	called by test_reference_match.main	—
def _proof(ref, amount, pid = 'p1')	(proof)	—	called by test_reference_match.main	—
def main()	(main)	agent/gate.decide orchestrator._norm_ref orchestrator._reference_pairs statement_parser.extract_reference test_reference_match._inv test_reference_match._proof test_reference_match._txn	—	—

test_myinvois_map.py — Self-check script for MyInvois UBL->invoice mapping (no top-level functions — runs inline).

No top-level functions extracted (file has no module-level def).

test_accounting_map.py — Self-check script for accounting-connector mapping (no top-level functions — runs inline).

No top-level functions extracted (file has no module-level def).

7. Pages

page.tsx — Root route ("/"): shows the marketing landing page when signed out, or the reconciliation dashboard when signed in.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Page ()	Renders LandingPage when there is no session, otherwise DashboardPage.	useAuth components/LandingPage components/DashboardPage	renders LandingPage or DashboardPage based on auth state	component

layout.tsx — Root app layout: loads fonts, forces dark theme pre-paint, and wraps every route in ThemeProvider/AuthProvider/AppShell.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
RootLayout ({children})	Root HTML layout: loads Inter/JetBrains Mono fonts, injects a pre-paint dark-theme script, wraps children in ThemeProvider/AuthProvider/AppShell.	ThemeProvider AuthProvider AppShell next/font/google	sets html/body classes injects no-flash dark theme inline script	component

login/page.tsx — /login: email+password sign-in form against Supabase Auth.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
LoginPage ()	Sign-in page with corporate email/password fields inside a two-pane auth layout.	useRouter components/ui/Field components/ui/Button components/MeshBackground	sets isLoading/error/form state	component
handleSignIn (e: React.FormEvent)	Submits the sign-in form via supabase.auth.signInWithPassword and pushes to / on success.	supabase.auth.signInWithPassword	sets error/isLoading state redirects to /	handler

signup/layout.tsx — Layout wrapper for the /signup route segment (no html/body, just a container div).

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
SignUpLayout ({children})	Wraps signup route children in a plain div (no html/body — inherited from root layout).	—	wraps children in div.signup-container	component

signup/page.tsx — /signup: workspace onboarding form (company name, registration no, email, password) that calls supabase.auth.signUp.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
SignUpPage ()	Workspace signup form (company name, registration number, email, password) with a success state.	useRouter components/ui/Field components/ui/Button components/MeshBackground	sets isLoading/success/error/form state	component
handleSignUp (e: React.FormEvent)	Calls supabase.auth.signUp with company metadata; redirects to / if a session comes back, else asks for email confirmation.	supabase.auth.signUp	sets success/error/isLoading state redirects to / after 1s on success	handler

uploads/page.tsx — /uploads: tabbed page (Bank statements / Invoices / Payment proofs) for account management, file uploads, connector imports, and bank-feed sync.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
UploadsPage ()	Default export: wraps UploadsInner in a Suspense boundary (required for useSearchParams).	React.Suspense	—	component

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
UploadsInner ()	Main uploads UI: segmented tabs for Bank statements / Invoices / Payment proofs, deep-linked via ?tab=, hosting account management, uploads, and connector sync.	useAuth useSearchParams useToast	renders active tab content syncs tab state with ?tab= query param	component
fetchAccounts ()	Loads bank_account rows for the tenant and defaults selectedAccount to the primary (or first) account.	reads: bank_account	sets accounts/selectedAccount state	handler
addAccount ()	Validates the new-account form, clears other primaries if needed, then inserts a bank_account row for the tenant.	reads/writes: bank_account useAuth (smeld)	writes: bank_account shows toast resets form and closes it refetches accounts	handler
setPrimary (accountId: string)	Clears is_primary on all of the sme's accounts then sets it on the given account.	writes: bank_account	writes: bank_account shows toast refetches accounts	handler
deleteAccount (accountId: string)	Cascading delete: an account's statements, their transactions, and any reconciliation_match rows on those transactions, reverting affected invoices to unmatched and re-promoting a primary if none remains.	reads/writes: bank_statement, bank_transaction, reconciliation_match, invoice, bank_account	writes: bank_account, bank_statement, bank_transaction, reconciliation_match, invoice shows toast refetches accounts and ledger	handler
fetchLedger ()	Loads bank_transaction rows joined with bank_statement(account_id) into the ledger table, newest first.	reads: bank_transaction, bank_statement	sets ledger state	handler
fetchPending ()	Loads invoice rows with status pending or unmatched.	reads: invoice	sets invoices state	handler
fetchProofs ()	Loads payment_proof rows ordered by uploaded_at descending into the proofs list.	reads: payment_proof	sets proofs state	handler
uploadStatement (files: FileList)	Posts the selected file (plus account_id) to the statement upload endpoint, then refetches the ledger.	POST /api/upload/statement useAuth (authHeaders)	sets stmtStatus state shows toast refetches ledger after 2.5s	handler
fetchConnectors ()	Reads myinvois_credential and accounting_credential to determine which import connectors are configured/usable.	reads: myinvois_credential, accounting_credential	sets connectors state	handler
runImport (key: 'myinvois' 'autocount' 'sql')	Calls the MyInvois or accounting sync endpoint for the chosen connector and refreshes pending invoices.	POST /api/myinvois/sync POST /api/accounting/sync? provider= useAuth (authHeaders)	shows toast refetches pending invoices sets syncing state	handler
syncFeed ()	Calls the Finverse bank-feed sync endpoint and refreshes both accounts and the ledger.	POST /api/bankfeed/sync useAuth (authHeaders)	shows toast refetches accounts and ledger sets feedSyncing state	handler
uploadInvoice (files: FileList)	Posts the selected file to the invoice upload endpoint, then refetches pending invoices.	POST /api/upload/invoice useAuth (authHeaders)	sets invStatus state shows toast refetches pending invoices after 1.8s	handler
uploadProof (files: FileList)	Posts the selected file to the payment-proof upload endpoint, then refetches proofs.	POST /api/upload/payment_proo f useAuth (authHeaders)	sets proofStatus state shows toast refetches proofs after 2.5s	handler
errDetail (data: any)	Extracts a readable error string from a FastAPI error response, handling both plain-string detail and 422 validation-array detail shapes.	—	—	util
invoice polling effect useEffect keyed on [invoices, fetchPending]	Re-polls fetchPending every 3s, for up to 60s, while any invoice still lacks both invoice_number and error_message (still parsing).	fetchPending	schedules repeated fetchPending calls	handler

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
proof polling effect useEffect keyed on [proofs, fetchProofs]	Re-polls fetchProofs every 3s, for up to 60s, while any proof's status is neither completed nor failed.	fetchProofs	schedules repeated fetchProofs calls	handler

audit/page.tsx — /audit: review queue of reconciliation_match rows the engine left as pending_review; Resolve or Reject each.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
AuditLogPage ()	Lists reconciliation_match rows with match_status=pending_review and lets the user resolve or reject each with an inline confirm.	useToast reads: reconciliation_match lib/matchActions.rejectMatch	renders exceptions table sets exceptions/loading/confirming/busy state	component
fetchExceptions ()	Loads pending_review reconciliation_match rows joined with invoice into the Exception list.	reads: reconciliation_match	sets exceptions state shows toast on error	handler
apply (e: Exception, action: 'resolve' 'reject')	Resolve sets match_status=manual (and invoice.status=matched); Reject delegates to the shared rejectMatch helper.	reads/writes: reconciliation_match lib/matchActions.rejectMatch	writes: reconciliation_match writes: invoice removes row from exceptions list shows toast	handler
fmtVariance (amount: number, currency: string)	Formats a signed variance amount with its currency code for display.	—	—	util
isHighRisk (e: Exception)	Flags an exception as high risk when confidence < 0.5 or variance magnitude > 5%.	—	—	util

history/page.tsx — /history: ledger of resolved matches (auto/manual/rejected) with filters and send-to-review/reject corrections.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
HistoryPage ()	Ledger of resolved reconciliation_match rows (auto/manual/rejected) with a segmented filter and send-to-review/reject actions.	useToast reads: reconciliation_match lib/matchActions.sendToReview lib/matchActions.rejectMatch	renders match history table sets rows/loading/filter/confirming/busy state	component
fetchRows ()	Loads reconciliation_match rows with status in (auto, manual, rejected) joined with invoice.	reads: reconciliation_match	sets rows state shows toast on error	handler
act (row: MatchRow, action: 'review' 'reject')	Sends a match back to pending_review or rejects it, via the shared matchActions helpers.	lib/matchActions.sendToReview lib/matchActions.rejectMatch	writes: reconciliation_match (via helpers) updates rows state shows toast	handler
SourcePill ({status: MatchStatus})	Renders a StatusPill labeled Automated/Manual/Rejected based on match status.	components/ui/StatusPill	—	component
fmtVariance (amount: number, currency: string)	Formats a signed variance amount with its currency code for display (same logic as audit page's copy).	—	—	util

settings/page.tsx — /settings: "Integrations" page for connecting MyInvois, the Finverse bank feed, and mock AutoCount/SQL Account connectors.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
SettingsPage ()	Integrations page rendering BankFeedCard, MyInvoisCard, and two AccountingCard instances (AutoCount, SQL Account).	components/ui/PageHeader	renders integration cards	component
StatusChip ({configured, isReal})	Shared header pill showing Not configured / Mock / Connected for a connector card.	components/ui/StatusPill	—	component

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
MyInvoisCard ()	Loads and saves the tenant's myinvois_credential (environment, TIN, client id/secret).	useAuth reads/writes: myinvois_credential	writes: myinvois_credential shows toast sets configured/environment/tin/clientId/clientSecret state	component
BankFeedCard ()	Lists active bank_feed_link consents, starts a new Finverse Connect flow, and toasts the outcome of the redirect-back (?linked=).	useAuth reads: bank_feed_link POST /api/bankfeed/link	writes: bank_feed_link (disconnect) shows toast reads ?linked= query param and clears it	component
connect () inside BankFeedCard	Calls POST /api/bankfeed/link and full-page-redirects the browser to the returned Finverse link_url.	POST /api/bankfeed/link useAuth (authHeaders)	redirects window.location to link_url shows toast on failure sets connecting state	handler
AccountingCard ({provider, title, description})	Generic mock-only connector card (AutoCount/SQL Account) that reads and upserts accounting_credential for the given provider.	useAuth reads/writes: accounting_credential	writes: accounting_credential shows toast sets configured/loading/saving state	component

8. Components

components/DashboardPage.tsx — Reconciliation dashboard — account-scoped funds/matched/unmatched stats, match-accuracy ring, source feed, matches table, anomaly panel, and the run-reconciliation flow with live activity trace.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
curateLine (r: {event_type?, message?, metadata?}): LogLine null	Maps a whitelisted set of reconciliation_log event types to one curated sentence for the activity drawer, dropping everything else.	—	UI: activity drawer log lines	util
myr (n: number) => string	Module-level MYR currency formatter (Intl.NumberFormat, no fraction digits).	—	—	util
DashboardPage ()	Main reconciliation dashboard: account toggle, funds-reconciled/matched/unmatched stats, accuracy ring, documents panel, anomalies panel, source feed and matches tables, activity drawer.	useAuth reads: bank_transaction reads: bank_statement reads: invoice reads: payment_proof reads: bank_account reads: reconciliation_match reads: reconciliation_job reads: reconciliation_log POST /api/reconcile GET /api/job-status next/navigation useRouter	navigates to /uploads UI state: matches/stats/logs/anomalies	component
renderTrace (jobId: string) => Promise<void>	Fetches reconciliation_log rows for a job (ordered by created_at) and replaces the activity log with curated lines via curateLine.	reads: reconciliation_log	UI state: logs	handler
loadAnomalies () => Promise<void>	Finds the most recent reconciliation_job and loads its anomaly_detected log entries to populate the anomalies panel.	reads: reconciliation_job reads: reconciliation_log	UI state: anomalies	handler
loadStats () => Promise<void>	Loads all bank transactions (joined to bank_statement for account_id), pending/unmatched invoice count, proof count, and bank accounts for the documents/source-feed panels.	reads: bank_transaction reads: bank_statement reads: invoice reads: payment_proof reads: bank_account	UI state: accounts/allTx/stats	handler
loadReconciliationState () => Promise<void>	Loads accepted (auto/manual) reconciliation_match rows plus unmatched invoice count to build the matches table, per-account funds-reconciled totals, and match-accuracy stats.	reads: reconciliation_match reads: invoice	UI state: matches/matchFunds/jobStats/hasRun	handler
handleReconcile () => Promise<void>	POSTs /api/reconcile then polls /api/job-status every 2s, streaming the run's trace via renderTrace until the job completes, fails, or is cancelled, then reloads stats/matches/anomalies.	POST /api/reconcile GET /api/job-status useAuth (authHeaders)	UI state: isRunning/logs/matches/hasRun	handler
Stat (props: {icon, label, value})	Small labeled stat tile with icon and skeleton loading state, used for transaction/matched/unmatched counts.	—	—	component

components/AppShell.tsx — Top-level app chrome — auth-gated routing, top navigation (Home/Upload dropdowns, profile menu), mobile nav, and shared MeshBackground/ToastProvider wrapper for all protected pages.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
NavDropdown (props: {id, label, active, items, openId, setOpenId, onNavigate?})	Desktop dropdown menu of nav links; open/closed state is owned by the parent so only one menu is open at a time.	next/link	UI state: openId (via setOpenId)	component

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
TopNav ()	App header: logo, Home/Upload nav dropdowns, desktop profile menu (MyInvois settings link, change account, log out), and a mobile hamburger panel; closes menus on outside click, Escape, or route change.	useAuth (companyName, signOut) usePathname useRouter	navigates to /login on sign out UI state: openId/mobileOpen	component
handleSignOut () => Promise<void> (inside TopNav)	Calls the auth context's signOut then navigates to /login.	useAuth (signOut)	navigates to /login	handler
handleChangeAccount () => Promise<void> (inside TopNav, alias of handleSignOut)	"Change account" action; since one user maps to one workspace today, it just signs out to the login screen.	—	navigates to /login	handler
MobileSection (props: {title, items, onNavigate})	Renders a titled group of nav links inside the mobile dropdown panel.	next/link	—	component
AppShell (props: {children: React.ReactNode})	Top-level shell: redirects unauthenticated users to /login and authenticated users away from auth routes, renders auth/public routes bare, and wraps protected routes in ToastProvider + MeshBackground + TopNav.	useAuth (session, loading) usePathname useRouter MeshBackground ToastProvider (ui/Toast)	navigates to /login or / UI: wraps all protected pages	component

components/LandingPage.tsx — Marketing landing page (dark 'tech' theme) — hero, how-it-works steps, pain-point agitation section, old-way/new-way comparison, and an interactive ROI calculator.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
useInView (threshold = 0.2)	IntersectionObserver-based scroll-reveal hook that fires once and treats prefers-reduced-motion users as already revealed with no transition.	—	—	hook
Reveal (props: {children, delay?, className?})	Wraps children with a fade+lift transition once scrolled into view (staggered by delay), backed by useInView.	useInView	—	component
RoiCalculator ()	Interactive slider for weekly manual-reconciliation hours that computes illustrative hours/cost saved per year using hardcoded assumptions, with a signup CTA.	next/link	navigates to /signup (link)	component
LandingPage ()	Marketing landing page: navbar, hero with mount-triggered fade-in CTA, "how it works" step list, pain-point cards, old-way vs new-way comparison, and the ROI calculator, over a MeshBackground.	MeshBackground next/link	navigates to /login, /signup (links)	component

components/MeshBackground.tsx — Shared low-poly geometric background visual (SVG mesh + glow) reused across landing/auth/app pages at three intensity variants, plus a landing-only cursor glow.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
PolyMesh (props: {variant: 'landing' 'auth' 'app'})	Renders the low-poly SVG triangle mesh, purple/blue radial gradient washes, and glow edge lines, styled per-variant from the CFG table.	—	—	component
CursorGlow ()	Landing-only cursor-following radial glow that updates CSS custom properties directly via a rAF-throttled pointermove listener; disabled under prefers-reduced-motion.	—	DOM style: CSS custom properties --mx/--my	component
MeshBackground (props: {variant: 'landing' 'auth' 'app'})	Exported entry point that positions PolyMesh (fixed vs absolute depending on variant) and adds CursorGlow only for the landing variant.	PolyMesh CursorGlow	—	component

9. UI kit

components/ui/ActivityDrawer.tsx — Bottom-docked collapsible activity/log console showing processing trail lines with a status bar summary.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
ActivityDrawer ({ lines, running?, title? })	Renders a fixed bottom bar showing the latest log line and a running indicator, expandable to a scrollable log of all lines colored by level.	lucide-react cn	manages open/collapsed state auto-scrolls log to bottom on new lines	component

components/ui/Button.tsx — Base button component with variant/size styling and a built-in loading spinner state.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Button (props: ButtonProps extends ButtonHTMLAttributes)	Styled button supporting variant (primary/secondary/ghost/danger), size (sm/md), an icon slot, and a loading state that swaps the icon for a spinner and disables the button.	lucide-react cn	N/A — pure render, disables itself when loading	component

components/ui/Dropzone.tsx — Click-or-drag file upload target with idle/uploading/done/error visual states.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Dropzone ({ accept, onFiles, status?, title, hint, message?, disabled? })	Clickable/drag-and-drop file picker that shows idle/uploading/done/error visuals and calls onFiles with the selected/dropped FileList.	lucide-react cn	manages internal drag-over state triggers hidden file input	component

components/ui/EmptyState.tsx — Generic empty/placeholder state block (icon, title, description, optional action).

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
EmptyState ({ icon?, title, description?, action?, className? })	Centered placeholder block rendering an optional icon, title, description, and action node.	cn	N/A — pure render	component

components/ui/Field.tsx — Labeled text input with hint/error text and accessible aria wiring.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Field (props: FieldProps extends InputHTMLAttributes, incl. label, hint?, error?, icon?)	Labeled input field with optional leading icon, hint text, or error text, wired with aria-invalid/aria-describedby.	React useld cn	N/A — pure render (controlled via passed input props)	component

components/ui/PageHeader.tsx — Page-level header with title, optional description, and an optional right-aligned action slot.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
PageHeader ({ title, description?, action? })	Renders a page title/description header row with an optional right-aligned action slot.	—	N/A — pure render	component

components/ui/Panel.tsx — Card/panel container plus a PanelHeader subcomponent for titled sections with icon/action.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Panel (props: HTMLAttributes<HTMLDivElement>)	Bordered/rounded card container div that passes through className and other div props.	cn	N/A — pure render	component
PanelHeader ({ title, icon?, action?, className? })	Header row for a Panel with an optional icon, title text, and a right-side action node.	cn	N/A — pure render	component

components/ui/RingProgress.tsx — Accessible SVG ring progress indicator (0-100) with center label/sublabel.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
RingProgress ({ value, size?, stroke?, label?, sublabel? })	Draws an SVG circular progress ring from a 0-100 value with a centered label/sublabel and animated stroke-dashoffset transition.	—	N/A — pure render, derives geometry from value prop	component

components/ui/SegmentedControl.tsx — Accessible tablist-style segmented tab switcher with roving arrow-key focus.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
SegmentedControl ({ items, value, onChange, className?, 'aria-label'? })	Renders a tablist of segmented buttons with active-item styling, calling onChange on click and supporting arrow-key roving focus between tabs.	cn	manages a ref array for focus targets; drives external selected-tab state via onChange	component

components/ui/Skeleton.tsx — Loading-state skeleton block plus SkeletonRows helper for table loading placeholders.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
Skeleton ({ className? })	Renders a single pulsing placeholder block for loading states.	cn	N/A — pure render	component
SkeletonRows ({ rows?, cols? })	Renders a <tbody> of skeleton table rows/cells (first column wider) for table loading states.	Skeleton cn	N/A — pure render	component

components/ui/StatusPill.tsx — Small colored pill for status/tone display (success/warning/danger/info/neutral) with default or custom icon.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
StatusPill ({ tone, children, icon?, className? })	Renders a rounded pill with tone-based background/text color and an icon (default per tone, custom override, or none if icon is null).	lucide-react cn	N/A — pure render	component

components/ui/Table.tsx — Table primitives: horizontal-scroll wrapper, table, sticky header cell, data cell, and row.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
TableScroll ({ className?, children })	Wraps children in a horizontally scrollable div so wide tables don't break layout.	cn	N/A — pure render	component
Table ({ className?, children })	Base <table> element with left-aligned, border-collapsed default styling.	cn	N/A — pure render	component
Th ({ className?, align?, children? })	Sticky, uppercase-styled table header cell with left/right/center alignment options.	cn	N/A — pure render	component
Td ({ className?, align?, mono?, children?, colSpan? })	Table data cell with alignment, optional monospace/tabular-number styling, and colSpan support.	cn	N/A — pure render	component
Tr ({ className?, hover?, children })	Table row with a bottom border and an optional hover-highlight style.	cn	N/A — pure render	component

components/ui/Toast.tsx — Toast notification system: ToastProvider (context + rendered stack, auto-dismiss after 5s) and useToast hook to trigger toasts.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
ToastProvider ({ children })	Context provider that renders app children plus a fixed bottom-right toast stack; each toast auto-dismisses after 5 seconds or can be dismissed via its close button.	React context lucide-react cn	manages toast queue state (add via toast(), remove via timeout or dismiss click)	component

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
useToast () => { toast }	Reads the ToastContext to expose a toast(...) function for triggering a notification from anywhere under ToastProvider.	ToastContext (React context)	reads/exposes the ToastProvider's toast-queue setter indirectly via toast()	hook

components/ui/cn.ts — One-line classnames joiner utility (filters falsy values, no external dependency).

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
cn (...parts: Array<string false null undefined>) => string	Joins truthy class-name strings with a space, filtering out false/null/undefined entries.	—	N/A — pure function	util

components/ui/index.ts — Barrel file re-exporting the whole UI kit.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
(barrel re-exports) N/A — no logic, only export statements	Re-exports cn, Button, Field, Panel/PanelHeader, StatusPill (+ Tone), Table/TableScroll/Th/Td/Tr, Skeleton/SkeletonRows, EmptyState, RingProgress, Dropzone (+ UploadStatus), SegmentedControl (+ SegmentItem), ToastProvider/useToast, PageHeader, and ActivityDrawer (+ LogLine) from their individual files.	all other components/ui files	N/A — no state	util

10. lib & contexts

lib/AuthContext.tsx — Supabase Auth session provider — tracks session/loading, resolves the signed-in user's sme workspace (smeld, companyName), and exposes signOut + JWT auth headers for backend calls.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
AuthProvider props: { children: React.ReactNode }	Provides session, smeld, companyName, loading, signOut, and authHeaders via React context, loading the initial session then subscribing to auth state changes.	lib/supabaseClient @supabase/supabase-js reads: sme	auth session state smeld/companyName context state	context-provider
useAuth no params; returns AuthValue	Reads AuthContext and throws if called outside an AuthProvider.	AuthContext (AuthProvider)	none — read-only accessor	hook

lib/matchActions.ts — Frontend helpers for /audit and /history to revert or reject a reconciliation_match, keeping invoice status and bank_transaction.is_matched in sync and logging a teaching signal on reject.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
sendToReview (m: MatchRef) => Promise<{ error: string null }>	Sets a reconciliation_match back to match_status='pending_review' and the linked invoice's status to 'partial', leaving the transaction still linked.	lib/supabaseClient	writes: reconciliation_match writes: invoice	util
rejectMatch (m: MatchRef) => Promise<{ error: string null }>	Sets match_status='rejected', unmarks the invoice as unmatched and the transaction as unlinked, and best-effort logs a match_rejected event with invoice/transaction/variance metadata for the learned-memory blocklist.	lib/supabaseClient	writes: reconciliation_match writes: invoice writes: bank_transaction writes: reconciliation_log	util

lib/supabaseClient.ts — Creates the single shared browser Supabase client from public env vars, used by all direct frontend Supabase reads/writes.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
supabase const supabase = createClient(url, anonKey)	Single shared Supabase browser client instance built from NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY, used app-wide; RLS scopes queries to the session JWT.	@supabase/supabase-js env: NEXT_PUBLIC_SUPABASE_U RL env: NEXT_PUBLIC_SUPABASE_A NON_KEY	all frontend Supabase reads/writes	util

ThemeContext.tsx — Light/dark theme provider that syncs React state to the class the no-flash inline script already applied to <html>, and persists user changes to localStorage.

FUNCTION (SIGNATURE)	DESCRIPTION	DEPENDS ON	AFFECTS	KIND
ThemeProvider props: { children: React.ReactNode }	Syncs React theme state to the dark/light class already set on <html> by the no-flash inline script, and provides setTheme/toggle that update the DOM class, color-scheme style, and localStorage.	document.documentEleme nt localStorage	theme context state html.dark class localStorage tf-theme key	context-provider
useTheme no params; returns { theme, toggle, setTheme }	Reads ThemeContext; returns the default context value (light theme, no-op functions) if used outside ThemeProvider since no guard throws.	ThemeContext (ThemeProvider)	none — read-only accessor	hook